By Their Fruits You Will Know Them: Comparing Formalizations of Law by the Decisions They Encode

Julius Vernie
Technical University of Munich (TUM)
julius.vernie@tum.de

Matthias Grabmair
Technical University of Munich
matthias.grabmair@tum.de

Abstract

Formalizing legal provisions promises machine-accessible law and automated legal reasoning, and recent LLMs make it tempting to generate such formalizations directly from statutory text. However, any formalization makes implicit interpretive choices whose consequences are hard to anticipate, especially if an LLM is the author. We present a method for systematically comparing different formalizations of the same legal provision by their inferences on individual cases. Given multiple formalizations of a provision, we match them at the node level, derive a shared interface for each pair from the matching, and use a SAT solver to enumerate the edge cases on which any two formalizations disagree. Selected edge cases are then verbalized into concrete factual scenarios that a legal expert can examine and act on. We apply our method to formalizations of ten EU provisions generated by nine frontier LLMs. We find that structural alignment does not predict behavioral divergence. Formalizations whose matching aligns most of their structure still disagree on many cases. The verbalized cases reveal qualitatively distinct types of disagreement, including divergences that mirror genuine controversies in the legal commentary.

1 Introduction

Concerns about the ambiguity of natural-language legal text and calls to represent the law in a formalized, unambiguous manner go back at least to the early days of symbolic logic (Allen, 1957). The pay-off, if achievable, would be substantial: a formal representation makes the law machine-accessible and enables automated legal decisions in a broad range of settings. The idea has regained traction with the rise of LLMs, which possess enough contextual understanding to — at least in principle — produce such formalizations directly from statutory text (Janatian et al., 2023; Graus, 2026).

However, the call for formalization is to be taken with caution. Any formalization makes implicit interpretive choices, and foreseeing the full consequences of those choices is hard: a sound-looking formalization can yield an unjust verdict in a future case its drafter did not anticipate. The problem is even more pressing when the formalization is produced by an LLM, whose interpretive choices are opaque to the reviewer.

We therefore address the question of how to make different formalizations of the same legal provision systematically comparable with respect to their outputs. We present a method that first matches multiple formalizations of a provision against one another, then uses SAT-based analysis to enumerate every input assignment on which any two of them disagree. We apply the method to nine LLM-generated formalizations of ten EU provisions across the GDPR, the AI Act, the DMA, and other instruments. We find that outcome disagreement persists even between formalizations whose matching aligns most of their structure, so structural comparison alone does not reveal it. We perform a case study and show that verbalization helps to surface deeply rooted divergences. For example, we find that the models split evenly into two camps on whether an AI system performing biometric categorization by race for law enforcement would be prohibited according to Article 5 AI Act, an interpretive divide that is also visible in the legal commentary on the provision.

Specifically, we make the following contributions:

(i) a framework for comparing different formalizations of the same legal provision by cases on which they disagree;

(ii) a verbalization method that turns SAT-derived edge cases into concrete factual scenarios for human review;

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 45830–45855
October 24-29, 2026 ©2026 Association for Computational Linguistics

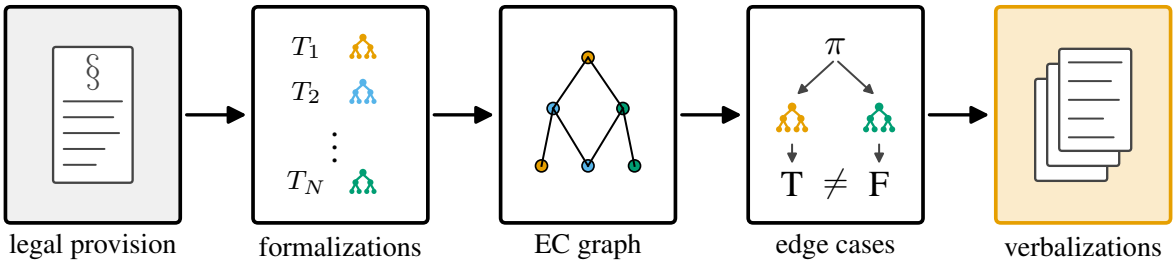


Figure 1: Analysis pipeline. From a legal provision, N LLMs each produce a formalization; an LLM matches nodes across the formalizations into an equivalence-class (EC) graph; pairwise SAT analysis enumerates edge cases; an LLM verbalizes them for legal review.

(iii) an empirical study of nine frontier LLMs on ten EU provisions, revealing systematic divergences that purely structural comparison cannot detect.[1]

2 Related Work

2.1 Legal Formalizations

The approach to represent laws and legal provisions in formal algorithmic ways is not new. Notable early contributions include normalized legal drafting (Allen and Engholm, 1978) and the direct transcription to Prolog (Sergot et al., 1986). More recently, several approaches for dedicated legal programming languages appeared, such as Catala (Merigoux et al., 2021) for computational law, Stipula (Crafa et al., 2023) for legal contracts, and L4 (Governatori and Wong, 2023) with defeasible legal-reasoning semantics; Ma and Wilson (2021) discuss the syntax such a representation should take.

While the process of formalizing the law was originally human-driven, the rise of LLMs and their fast adoption for general coding tasks quickly raised the question of whether they can applied to legal formalization as well. Notable work here include Janatian et al. (2023), who explored the potential of LLMs for legal formalization and Dal Pont et al. (2025), who used LLMs to represent EU regulation in knowledge graphs.

Nguyen et al. (2026) generate GDPR formalizations as condition–exception rule trees with LLM agents and validate them through a panel of verifier agents and human review, reporting that failures stem mostly from abstraction choices rather than logical errors. While their method validates a single formalization against expert judgment, ours compares several formalizations to each other and needs no reference encoding. The authors also map a natural-language scenario on the generated formalizations, however do not generate edge-case scenarios from logical differences.

2.2 Matching

The identification of corresponding concepts across heterogeneous structured representations is studied as *ontology matching*, with nearly two decades of benchmarking through the Ontology Alignment Evaluation Initiative (Euzenat et al., 2011). Matchers exploit the signals a curated ontology supplies, namely the labels of its entities, the structure of its class and property hierarchy, the instances populating its classes, and the axioms a reasoner can act on (Otero-Cerdeira et al., 2015). Established systems include LogMap (Jiménez-Ruiz and Cuenca Grau, 2011) and AgreementMakerLight (Faria et al., 2013).

With the rise of LLMs, prompt-based matchers have entered the field. Macilenti et al. (2024) report the limits of prompting alone, finding that a prompting recipe carried from the OAEI benchmarks to a larger, unpublished alignment loses most of its accuracy and fails on hierarchical correspondences, where one concept is broader or narrower than the other. They attribute the drop to the size of the candidate space and to the presence of such correspondences. Parciak et al. (2024) match schema elements from names and short descriptions alone, in a setting where instances are unavailable and labels are too cryptic for thesauri. Prompting one element against a set of candidates outperforms a string-similarity baseline on nearly every dataset they test, while prompting isolated pairs or entire cross-products does worse.

Our setting falls inside these limits. Fewer than a thousand nodes enter the matching of any sin-

[1] Experiment code and data are published at https://doi.org/10.5281/zenodo.22128911.

gle provision, and only equivalence relations are required. Additionally, an LLM can be provided with unstructured context for the matching. In our case, we want to supply the text of the provision to improve interpretation of the formalization. We therefore decide to use an LLM for the matching step.

2.3 Equivalence Checking

Checking whether two Boolean functions agree on every input is a well-studied problem. Prime implicants (Quine, 1952) are a compact way to summarize the cases in which two such functions disagree. Their main application has been in hardware design, where they are central to minimizing switching circuits and to checking whether two alternative chip designs implement the same function (Kuehlmann et al., 2002). Today, equivalence checking is largely handled by SAT and SMT solvers, descending from the DPLL procedure (Davis et al., 1962) and available in modern tools such as Z3 (de Moura and Bjørner, 2008).

Outside the legal domain, Li et al. (2024) compare multiple LLM formalizations of the same mathematical statement by symbolic equivalence, using agreement among the candidates as a selection signal. We share the premise that equivalence between independently generated formalizations is informative where no reference exists, but use it to localize disagreement rather than to select a winner. Additionally, unless some form of formal semantics and closed domain vocabulary is defined ahead of time, formalizations of realistically complex codified law usually have neither a theorem prover to appeal to nor a shared vocabulary to begin with, which is what the matching and interface steps supply.

Logical analysis of legal artifacts has appeared only sporadically. Allen (1957) uses a manually-crafted truth table to prove that a formalized version of a part of the U.S. Internal Revenue Code is equivalent to the actual law. Governatori et al. (2018) use defeasible deontic logic to formally represent a legal normative system and detect conflicts within it. Khoja et al. (2025) formalize contract clauses in decidable fragments of first-order logic and use SMT to detect internal inconsistencies. Adjacent work in business-process compliance analyses how regulatory changes affect formal process models (Barrientos et al., 2026).

Most recently, Graus (2026) generated LLM-based decision models and evaluated them against a human-crafted gold standard via outcome similarity, that is, the share of input combinations on which model and gold standard reach the same result. Our setup differs on three counts. First, we compare formalizations against one another rather than against a gold standard, and so make no assumption about correctness. Second, we find the input nodes for SAT analysis for each pair based on matching, rather than requiring a fixed set of inputs. Third, we surface specific edge cases as concrete factual scenarios (Section 4.4), rather than collapsing the comparison to an aggregate similarity score.

2.4 Verbalization

Making formal-verification artifacts accessible to non-experts is a recognized concern in adjacent fields. The literature on *counterexample explanation* surveyed by (Kaleeswaran et al., 2022) spans visual, trace-based, and template-driven approaches to translating model-checking counterexamples into more accessible forms. Moreira and Campos (2025) suggest going further and use LLMs to improve template-based counterexample explanations.

A separate line of work uses LLMs to *synthesize* concrete test scenarios from higher-level specifications in safety-critical domains, such as LeGEND (Tang et al., 2024) for autonomous-driving systems and (Al-Azzoni et al., 2025) for smart contracts. However, these works use test scenarios as inputs for system tests, instead of analyzing them as outputs.

3 Dataset

We study 10 legal provisions selected from EU law (Table 1). They span four domains: data protection (General Data Protection Regulation, GDPR), consumer law (Unfair Contract Terms Directive, UCTD; Unfair Commercial Practices Directive, UCPD), digital-market and AI regulation (Digital Markets Act, DMA; AI Act), and public procurement and state aid (Public Procurement Directive; Treaty on the Functioning of the European Union, TFEU). Provisions were selected according to three criteria:

(i) Complexity: the analysis of formalizations is only interesting for norms with potentially differing or erroneous interpretations. We therefore chose provisions with multiple nested conditions and non-

#	Provision	Legal consequence
1	Art. 6 GDPR	Data processing is lawful
2	Art. 9 GDPR	Processing of special-category data is prohibited
3	Art. 17 GDPR	Data subject has right to erasure
4	Art. 22 GDPR	Data subject has right not to be subject to automated decision-making
5	Art. 3 UCTD	Contractual term is unfair
6	Art. 6–7 UCPD	Commercial practice is misleading
7	Art. 3 DMA	Undertaking is designated as gatekeeper
8	Art. 5 AI Act	AI practice is prohibited
9	Art. 57 Proc. Dir.	Economic operator is excluded from procedure
10	Art. 107 TFEU	State aid is incompatible with the internal market

Table 1: Legal provisions and their central legal consequences. Numbers and colors identify provisions across all figures.

trivial interactions between them. This forces the LLM to make actual decisions about interpretation.

(ii) Self-containment: the provision should not refer extensively to other norms and should be mostly interpretable without having complete knowledge of the larger corpus of law.

(iii) Binary legal consequence: The provision must encode the decision of a clear, binary legal consequence to ensure that the root of every formalization is interpretable as a Boolean entitlement decision.

4 Methods

Figure 1 gives an overview of the full analysis pipeline. We provide a worked example in Appendix B to illustrate the process.

4.1 Formalization

A *formalization* of a legal provision is a rooted, labeled tree $T = (V, E)$. Each node $v \in V$ carries a natural-language label $\ell(v)$, and every non-leaf node additionally carries a logical operator $\mathrm{op}(v) \in \{\mathrm{AND}, \mathrm{OR}, \mathrm{NAND}, \mathrm{NOR}\}$. A formalization does not attempt to mirror the syntactic structure of the provision text. Instead, it captures the decision schema a legal expert would follow when assessing whether the provision applies to a concrete case, thereby making that reasoning structure explicit and machine-readable.

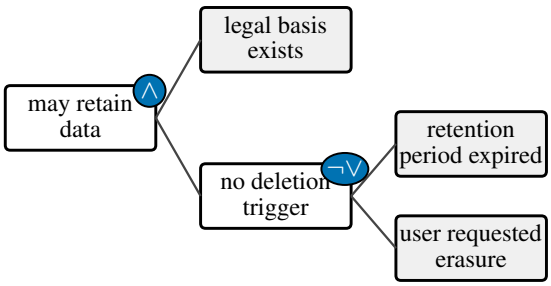


Figure 2: Illustration of a formalization. Internal nodes carry a logical operator and the root represents the legal consequence to be decided on.

For our study we instruct $N = 9$ different LLMs to produce a formalization of each provision; the full model list is given in Appendix A (Table 3). This yields a set of trees $\mathcal{T} = \{T_1, \ldots, T_N\}$ per provision.

4.2 Matching

To compare formalizations of the same provision we need to identify nodes across different trees that represent the same legal concept. We call this step *matching*.

Given N formalizations $\mathcal{T} = \{T_1, \ldots, T_N\}$ for a single provision, a matching groups all nodes across the N trees into *equivalence classes* (ECs); we write $\mathcal{E}$ for the full set of ECs. Each EC $e \in \mathcal{E}$ is a set of nodes drawn from distinct formalizations, that is, every formalization contributes at most one node to e. We interpret an EC as a legal concept present in the participating formalizations.

4.2.1 Choosing a Matching Method

Concept matching is the most complex step of the experiment. A correct matching requires understanding (1) the provision, (2) each formalization in its context, and (3) individual nodes in the context of their surrounding tree structure. Tree-edit-distance metrics and embedding-similarity baselines we tried in pilot experiments handle none of these requirements, and classical ontology matching systems are designed for settings that do not apply here (Section 2.2). Following Parciak et al. (2024), we therefore use an LLM for the matching step.

Direct accuracy measurement is, however, infeasible: no benchmark exists for matching formalizations of legal norms, and creating one would itself require resolving the very ambiguity we study. Two legal experts asked to match the same nodes are prone to disagree, since interpretation of the formalization, like interpretation of the law itself, is a matter of judgment. What we can assess is reliability: if the same model produces consistent results across independent runs, we have reason to trust it captures structural relationships rather than making arbitrary choices. We therefore conduct a consistency study across candidate models and select the most consistent one for all subsequent matching runs (Section 5.1).

Consistency, however, does not exclude systematic error. A matcher might consistently fail to match related legal concepts or match unrelated ones. We therefore use the consistency study only to select among candidate models and treat matching correctness as an open limitation.

4.3 Edge Case Analysis

Each formalization can be read as a Boolean function. The leaf nodes are input variables, internal nodes apply their operator to their children, and the root yields the legal consequence.

4.3.1 Edge Cases

Given two formalizations T_A, T_B of the same legal provision over a shared set of Boolean input variables $\mathbf{x} = (x_1, \ldots, x_n)$, let $f_A(\mathbf{x})$ and $f_B(\mathbf{x})$ denote their respective root functions. We characterize edge cases through *prime implicants* (Quine, 1952) of the disagreement function $f_A \oplus f_B$. A prime implicant is a partial assignment $\pi : S \to \{\texttt{true}, \texttt{false}\}$, $S \subseteq \{x_1, \ldots, x_n\}$, such that (i) every completion of the unassigned variables yields $f_A \oplus f_B = \texttt{true}$, and (ii) no proper subset of S satisfies (i). Each prime implicant captures a distinct, minimal pattern of disagreement between T_A and T_B. We use *prime implicant* and *edge case* interchangeably throughout.

4.3.2 The Interface Problem

A Boolean comparison requires a shared input vocabulary, but different formalizations rarely share the same leaf nodes: one may split a concept into three sub-conditions where another uses two. The matching step's EC graph supplies this vocabulary.

We define the *interface* $\mathcal{I}_{A,B} \subseteq \mathcal{E}$ as the set of deepest ECs to which both T_A and T_B contribute a node, and write $\mathcal{E}^{\geq \mathcal{I}}$ for the ECs lying on or above it: those in $\mathcal{I}_{A,B}$ together with those on a path from a root to a member of $\mathcal{I}_{A,B}$. Reading every EC above the interface as its operator applied to its children, and every interface EC as an atomic variable, gives a Boolean formula f_A (resp. f_B) over the interface variables. This is the analogue of cut-point equivalence checking in hardware verification (Kuehlmann et al., 2002). Appendix B works the construction through on a small example.

The *coverage* of T_x measures how much of the formalization lies on or above the interface:

$$\mathrm{cov}(T_x, \mathcal{I}_{A,B}) = \frac{|\mathcal{E}^x \cap \mathcal{E}^{\geq \mathcal{I}}|}{|\mathcal{E}^x|}, \tag{1}$$

with $\mathcal{E}^x$ the set of ECs to which T_x contributes a node. For a pair we report the geometric mean of the two,

$$\mathrm{cov}(T_A, T_B) = \sqrt{\mathrm{cov}(T_A, \mathcal{I}_{A,B}) \cdot \mathrm{cov}(T_B, \mathcal{I}_{A,B})}. \tag{2}$$

Low coverage means most of either tree is invisible to the Boolean comparison, so we restrict to $\mathrm{cov}(T_A, T_B) \geq 0.4$ throughout, justified empirically in Figure 4 (left panel).

The interface is defined pairwise; a joint interface across all N formalizations would be pulled up by any structural outlier and collapse to a few high-level concepts.

Two formalizations may encode the same concept with very different granularity (Appendix B). The interface locates the deepest level at which they still share a vocabulary, so whatever lies below it is cut off from the Boolean comparison. This is inherent to an outcome-based analysis: where the decompositions part, no shared vocabulary is left to compare over. We want to keep the method applicable to formalizations of any structure. We therefore impose no granularity requirement. To make explicit how informative a comparison is, we report coverage, that is, the fraction of each formalization the comparison takes in.

4.3.3 Edge Case Extraction

For each pair, we build f_A and f_B as Boolean formulas over the interface variables by recursively translating each formalization's operators. Edge cases are then extracted via a SAT-iterative search using Z3 (de Moura and Bjørner, 2008) until every input assignment on which f_A and f_B disagree is captured. Appendix C gives the procedure in detail.

4.3.4 Root Cause Identification

A prime implicant identifies *which* input conditions force disagreement, but not *where* in the formalizations it originates. Each EC above the interface induces one sub-formula per formalization: the part of that formalization rooted at it. A *root cause* of a prime implicant π is an EC whose two sub-formulas are forced to differ under every completion of π, while no child of it is, in either formalization. It is the deepest point at which the two formalizations are already committed to different answers.

4.4 Verbalization

Long lists of prime implicants are hardly interpretable by human reviewers. We therefore *verbalize* selected edge cases into concrete factual scenarios in natural language that a legal expert can evaluate. Verbalizing every edge case is, however, both computationally infeasible and legally redundant: most edge cases within a pair are minor variants of a few underlying disagreement patterns and equivalent constellations arise in multiple pairs. We therefore apply a selection pipeline that reduces the raw edge-case sets to a small set of *representative edge cases* per provision.

First, we discard pairs below the coverage threshold introduced in Section 4.3.2. We further drop pairs whose root operators differ between T_A and T_B, since such pairs disagree at the topmost level of the formalization and offer no fine-grained edge case worth verbalizing.

Second, for each surviving pair, we group the edge cases by their root causes (Section 4.3.4). Each group has a canonical *signature*: at every deepest root cause, the operator, negation, and child set on the side that concludes `true` and on the side that concludes `false`. Edge cases with the same signature share both where the disagreement originates and how the formalizations diverge there. They therefore describe the same kind of legal disagreement.

Third, within each signature class, we select the edge case with the fewest fixed variables as the *representative*: a shorter edge case stipulates fewer conditions and therefore yields a more general factual scenario. We cap the number of representative edge cases to 25 per provision, dropping the edge cases with the largest number of input values first.

Finally, each representative is passed to Claude Opus 4.6 together with the provision text, the two formalizations of the witnessing pair, the fixed variables and their required truth values, the remaining interface variables, and the root causes. The model returns a short description of a factual scenario.

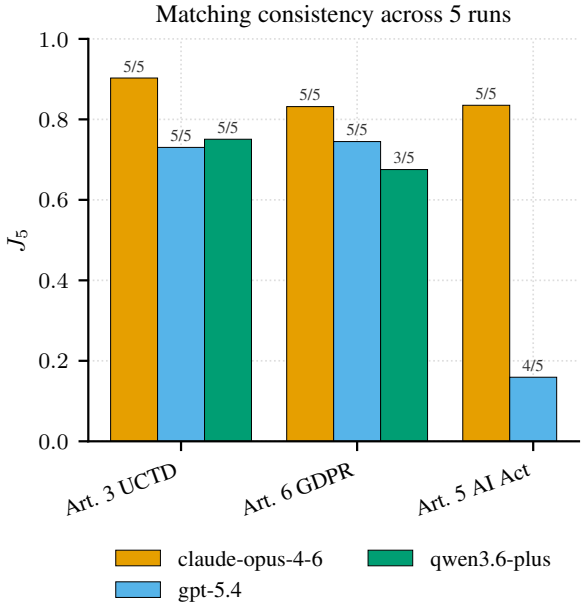


Figure 3: J_5 per (model, provision) across five independent matching runs. $k/5$ above each bar denotes number of runs that returned a schema-valid matching. Provisions are ordered by estimated complexity from low to high.

5 Results

5.1 Matching Consistency

We evaluate matching consistency by running each of three candidate models five times on each of three provisions and measuring the N-set Jaccard of the resulting matchings. Each run is summarized by its *co-membership pair set* $\mathcal{P}_i$ — the set of all atom pairs $\{(T, v), (T', v')\}$ placed in the same EC by run i, where each atom (T, v) identifies a node v of formalization T. The N-set Jaccard

$$J_N = \frac{\left|\bigcap_{i=1}^{N} \mathcal{P}_i\right|}{\left|\bigcup_{i=1}^{N} \mathcal{P}_i\right|}, \tag{3}$$

is the fraction of co-grouping decisions on which all N runs agree.

Only Claude Opus 4.6 delivers consistent matchings on all three provisions (Figure 3). GPT-5.4 produces schema-valid output in all but one run but disagrees with itself on Art. 5 AI Act. Qwen 3.6-plus already struggles on Art. 6 GDPR ($3/5$ valid

runs) and fails entirely on Art. 5 AI Act, the most complex provision tested. Note that for a group with $k < 5$ valid runs, J_N is computed over the valid runs only, so a group is scored on a smaller intersection. The scores of the weaker matchers are optimistic in this respect. We therefore adopt Claude Opus 4.6 for the matching step in all subsequent experiments.

5.2 Edge Case Analysis

We generate one formalization per model and legal provision, producing 90 formalizations and 360 pairs of formalizations. We then calculate the interface for every pair. Note that pairs are not independent data points. Each formalization enters eight of these pairs, and all nine formalizations of a provision are matched in a single step. The numbers below should therefore be interpreted as complete counts over the comparisons in our data, not estimates from a random sample.

Figure 4 (left) groups all pairs by their coverage, i.e. the share of the formalizations matched to each other and therefore available for edge-case analysis. Below a coverage of 0.2, 97% of pairs are equivalent; below 0.4, still 89%. This is the expected behavior: when both formalizations share almost no structure, there is little Boolean substance left to disagree over, so equivalence becomes the default rather than evidence of conceptual alignment. Above $\text{cov} \geq 0.4$ the equivalence rate flattens near 50% and stops moving meaningfully with further coverage. We therefore set a coverage threshold of 0.4 for the further analysis, leaving us with $N = 304$ pairs.

Figure 4 (right) shows the fraction of its pairs that are non-equivalent per model. The difference between models is large: Gemini 3.1 Pro disagrees with its peers on 71% of its pairs, Qwen 3.5 on only 32%. However, the rate of disagreement spans at least 50 percentage points across the ten provisions, with several swinging from 0% to 100% (not shown).

Figure 5 bins all pairs into coverage quartiles and plots the edge-case count per pair. The spread is enormous — three or more orders of magnitude in every bin — and the rank correlation between coverage and edge-case count is negligible ($\rho = +0.09$). The fraction of equivalent pairs decreases with increasing coverage, but only slightly. Even at the highest-coverage end, 42% of pairs are logically equivalent. Coverage thus characterizes *when* a behavioral comparison is informative, but not *how much* the two formalizations actually differ in their outcome. Even when models appear to agree on legal concepts, they can still diverge on many cases. Edge case analysis therefore adds a dimension of comparison that cannot be deduced from the matching information directly.

5.3 Verbalization

We apply the selection pipeline described in Section 4.4 and obtain 148 representative edge cases across the ten provisions. Table 2 shows two exemplary cases. The text of the provisions and visualizations of the formalizations can be found in Appendix D.

The first case demonstrates how an edge case can surface a genuine interpretive tension in the underlying statute. Eight frontier LLMs partition evenly on a highly sensitive provision concerning biometric racial categorization. Inspecting the formalizations, the disagreement traces to the structure of the carve-out clause: four models read *"categorizing of biometric data in the area of law enforcement"* as a free-standing exception that exempts any law-enforcement use, while the other four require the data to additionally have been lawfully acquired. The carve-out's syntax is in fact contested in the legal commentary: the European Commission's Guidelines treat law enforcement as one illustrative application of a unified exception (European Commission, 2025), whereas civil-society analyses argue that the statutory wording supports a narrower reading (AlgorithmWatch, 2025). We take no position on the correct interpretation; what matters for our purposes is that the pipeline localized the disagreement on a fact pattern where a legal expert can immediately recognize the underlying issue, rather than leaving it buried inside structurally similar trees.

The second case illustrates a disagreement that traces to an encoding error in one of the formalizations. Kimi K2.5 treats two conditions as both necessary for a term to count as not individually negotiated — the term must be pre-drafted *and* the supplier must fail to discharge the burden of proof — whereas Claude Opus and GLM-5.1 allow either condition to suffice. The latter reading is correct: Article 3(2)'s burden-of-proof rule is a procedural safeguard, not an additional substantive requirement, so Kimi's conjunctive encoding over-constrains the definition.

However, this case also shows a limit of edge-case analysis. The pipeline only surfaces what the

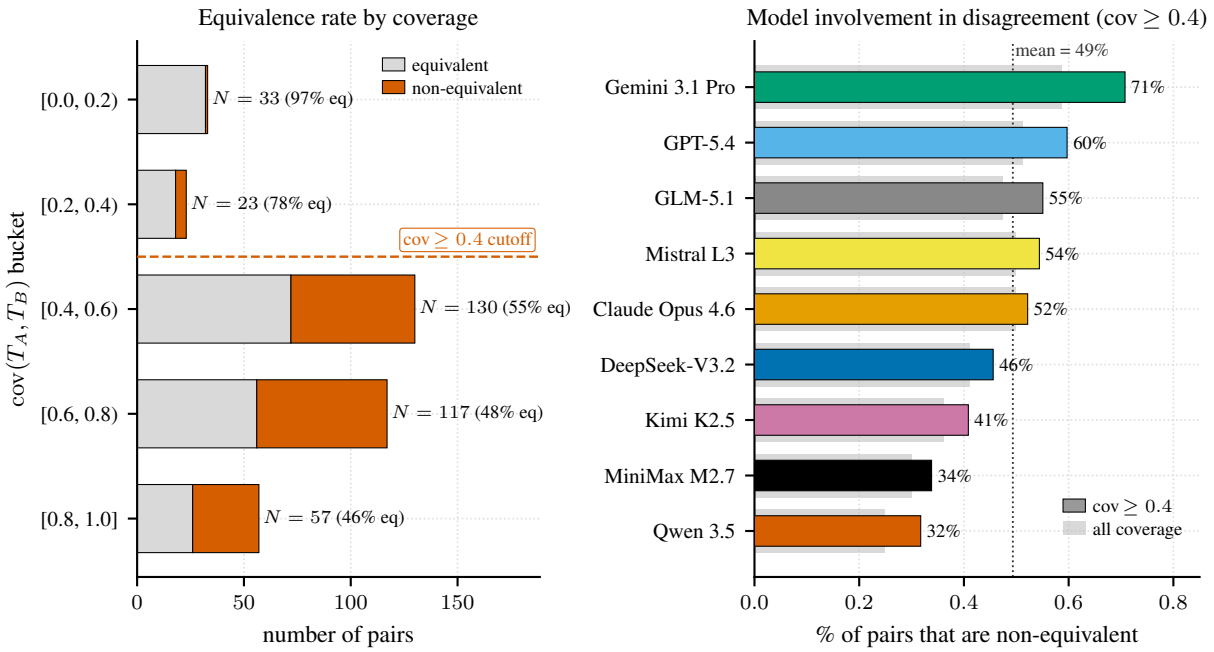


Figure 4: *Left:* Share of equivalent and non-equivalent formalization pairs per coverage bucket. Below the cov ≥ 0.4 cutoff (dashed), pairs are dominated by trivial equivalences whose interface barely crosses either formalization. *Right:* fraction of pairs involving each model that are non-equivalent for cov ≥ 0.4. Faded background bars repeat the same statistic without the coverage filter. Dotted line: overall mean across all pairs with cov ≥ 0.4.

Provision	**Scenario & question (LLM verbatim)**	**Model partition**
Art. 5 AI Act	SecureBio GmbH places on the EU market a biometric categorisation system that individually categorises natural persons based on their facial geometry to infer their race. The system is marketed specifically for use by national police forces to categorise biometric data in the area of law enforcement. The system is not used for labelling or filtering of lawfully acquired biometric datasets. SecureBio also markets the system as intended for medical reasons, specifically to assist forensic pathologists in identifying genetic predispositions linked to racial phenotypes for post-mortem medical examinations. *Is SecureBio's biometric categorisation system a prohibited AI practice under Article 5?*	**Yes, prohibited (4):** Claude Opus 4.6, Kimi K2.5, MiniMax M2.7, Mistral L3 **No, not prohibited (4):** Gemini 3.1 Pro, GPT-5.4, GLM-5.1, Qwen 3.5
Art. 3 UCTD	Maria joined FitLife Gym and signed a membership contract containing a clause that limits the gym's liability for personal injuries to €500, regardless of fault. The clause was composed during the sign-up meeting by the gym manager using improvised language rather than being taken from a template or pre-existing document. Maria had no opportunity to influence or modify the substance of this clause. The gym manager cannot provide evidence that the liability cap was individually negotiated with Maria. The clause shifts substantially all injury risk onto Maria while relieving the gym of obligations that would otherwise arise under general contract law. *Should this contractual term be regarded as unfair under Article 3?*	**Yes, unfair (2):** Claude Opus 4.6, GLM-5.1 **No, not unfair (1):** Kimi K2.5

Table 2: Two exemplary verbalized edge cases. The third column lists, for each side of the disagreement, the models whose formalization yields that conclusion on this fact pattern; models not listed produce a formalization that does not exhibit the relevant fork at the root cause and therefore do not contribute to either camp.

formalizations *disagree* on, while errors they share stay invisible. Here both GLM-5.1 and Kimi encode the pre-drafting test of Article 3(2) as the *definition* of "not individually negotiated", missing that Article 3(2) only establishes a default case and does not exclude other routes to the same conclusion (European Commission, 2019, § 1.2.2).

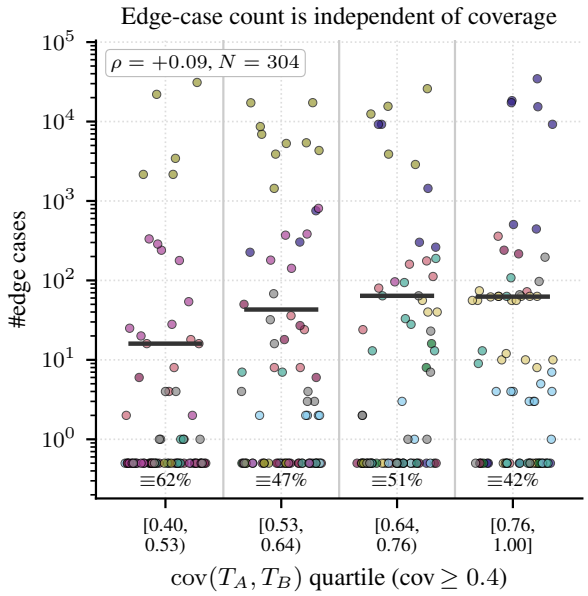


Figure 5: Edge-case count per pair across coverage quartiles. Dots are pairs, colored by provision. Dark ticks mark the median over non-equivalent pairs in each quartile. The $\equiv$ % annotation reports the fraction of logically equivalent pairs per quartile. Coverage and edge-case count are almost unrelated across these pairs ($\rho = +0.09$).

6 Discussion

Conceptual similarity does not reduce disagreement between formalizations, and higher coverage does not reduce it either. Equivalence checking is therefore a separate dimension of analysis that matching alone cannot capture. This finding is consistent with Graus (2026), who also reports cases of high structural similarity, but low outcome similarity.

Second, verbalization of edge cases exposes qualitatively distinct types of disagreement in a way a legal expert can adjudicate or at least localize at a glance. However, edge-case analysis and verbalization do not yield the full picture. Not all structural differences lead to a final difference in outcome; they might even cancel each other out. Further, a formalization that yields the legally correct result for a certain case is not necessarily structurally correct, as the Art. 3 UCTD case illustrates.

The method has many potential applications beyond analysis of LLM-generated formalizations. It may be applied, e.g. to compare human-authored encodings of the same statute, or one statute before and after a legislative amendment to discover unintended consequences.

During development of the method, we initially attempted to compute the interface jointly across all N formalizations, but quickly learned that this erases the very disagreements we want to study. Switching to pairwise SAT analysis, combined with the subsequent signature-based grouping, lets us make statements about N formalizations while preserving each pair's full internal structure.

In ongoing work we seek to integrate this method in an iterative LLM-driven formalization loop, using the verbalizations as intermediate feedback steps to gradually improve the quality of LLM formalizations in a given context.

7 Conclusion

We presented a method for systematically comparing formalizations of law by the decisions they encode. Given multiple formalizations of the same provision, we match them into an equivalence-class graph, derive a shared interface for each pair, and use SAT to enumerate the edge cases on which the two formalizations disagree. Selected edge cases are then verbalized into concrete factual scenarios. Applied to nine LLM-generated formalizations of ten EU provisions, we find that structural alignment does not predict behavioral divergence and that the verbalized cases expose qualitatively distinct types of disagreement, including divergences that mirror genuine controversies in the legal commentary.

Limitations

Several caveats constrain our results. First, the pipeline depends on matching nodes of different formalizations via an LLM, a step whose reliability we can assess but whose correctness we cannot (Section 4.2.1). We performed automated checks to assure the formal integrity of the matching results. In manual inspections we found the matchings to be mostly accurate, but we did sometimes observe nodes placed in separate ECs that, in our opinion, should have been matched. This may be partly an artifact of our requirement that the EC graph be loop-free. Quantifying the effect would require expert annotation of matchings, which we did not collect.

Second, we provide each LLM only the text of the provision to be formalized. No surrounding regulatory context such as related articles, recitals, official guidelines or case law is supplied. A legal expert formalizing the same article would routinely consult these sources to disambiguate terms, re-

solve cross-references, and check the encoded reading against established interpretation. The quality of our formalizations is therefore bounded by what each LLM already encodes about EU law in its training, which is unevenly distributed across instruments and known to under-represent recent or specialized material.

Third, we formalize single articles rather than full regulations. This keeps the task tractable but ignores the inter-article structure that gives statutory law much of its meaning: definitions in early articles (e.g. Art. 4 GDPR) propagate throughout, exceptions in one article modify another, and substantive provisions often depend on procedural conditions defined elsewhere. Each article is treated as if it were self-contained.

Lastly, verbalization hides several layers of complexity. The verbalization itself is generated by an LLM and may not faithfully encode the underlying prime implicant; we do not separately evaluate this fidelity, so an unfaithful scenario could elicit a verdict unrelated to the actual disagreement. Furthermore, formalizations can themselves be legally incorrect but still conclude correct outcomes in some cases. A reviewer's agreement with the outcome on a single scenario therefore is only weak evidence about the formalization as a whole. Verbalization-based review is therefore one signal among many rather than a definitive correctness test.

Ethical Considerations

Our pipeline analyzes LLM-generated formalizations of real, high-impact EU regulation. These formalizations are not validated for autonomous legal decision-making, and our central finding is that they can disagree substantially with one another. The pipeline is an audit aid for expert review, not a component of an automated legal system; a mis-formalization deployed without expert review could misassign rights or liability.

The analysis identifies where formalizations diverge, not which one is correct. The per-model statistic of Section 5.2 measures how often a model's formalizations differ from those of the other models, and must not be read as a quality ranking.

The verbalized scenarios concern fictional entities. No personal data was processed and no human subjects were involved.

Total LLM usage for the experiments amounted to approximately 5 M tokens.

Acknowledgments

This work was made possible by the funding of the Federal Ministry of Research, Technology and Space (Bundesministerium für Forschung, Technologie und Raumfahrt) funding line "KMU-Innovativ".

We thank Tilo Wend and Lado Sirdadze of our collaborator Rulemapping Group for extensive discussions and valuable insights about legal formalizations.

AI Use Disclosure

We used AI assistance throughout the research process to discuss and evaluate approaches and design decisions, but made all final decisions regarding the method and design of the experiment ourselves. We used AI agents to implement the experiment code. At several stages we checked the implemented features against the plan. We performed these checks partly manually, partly with AI assistance. Where we used AI to check code, the check ran in a separate session to ensure that the reviewing model evaluated the code against the plan without access to the context in which that code had been written. In Section 2, we used AI to identify relevant literature and to extract the relevant passages from cited papers. In Section 5, we used AI to identify and discuss potentially interesting aspects and patterns in the results data, but made all final decisions ourselves. In writing the paper, we used AI assistance to draft or revise individual paragraphs within a predefined scope. We checked, edited, and approved all generated text. We reviewed and verified the accuracy of all results. We take full responsibility for this work.

References

Issam Al-Azzoni, Saqib Iqbal, Taymour Al Ashkar, and Zobia Erum. 2025. Integrating model-driven engineering and large language models for test scenario generation for smart contracts. *Information*, 17(1):1.

AlgorithmWatch. 2025. Statement on Commission Guidelines on the AI Act. Public consultation submission.

Layman E. Allen. 1957. Symbolic logic: A razor-edged tool for drafting and interpreting legal documents. *The Yale Law Journal*, 66(6):833–879.

Layman E. Allen and Christopher Engholm. 1978. Normalized legal drafting and the query method. *Journal of Legal Education*, 29:380–412.

Marisol Barrientos, Karolin Winter, and Stefanie Rinderle-Ma. 2026. Impact analysis of regulatory requirement changes on business process compliance. *Information and Software Technology*, 194.

Silvia Crafa, Cosimo Laneve, Giovanni Sartor, and Adele Veschetti. 2023. Pacta sunt servanda: Legal contracts in Stipula. *Science of Computer Programming*, 225:102911.

Thiago Dal Pont, Federico Galli, Galileo Sartor, and Giuseppe Contissa. 2025. Lost in EU regulation? Don't worry, AI found the obligation. In *Proceedings of the International Conference on Artificial Intelligence and Law (ICAIL)*, pages 129–138.

Martin Davis, George Logemann, and Donald Loveland. 1962. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397.

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 337–340.

European Commission. 2019. Commission notice — guidance on the interpretation and application of Council Directive 93/13/EEC on unfair terms in consumer contracts. Official Journal of the European Union C 323/04, p. 4. CELEX:52019XC0927(01).

European Commission. 2025. Commission Guidelines on prohibited artificial intelligence practices established by Regulation (EU) 2024/1689 (AI Act). Communication C(2025) 5052 final, European Commission.

Jérôme Euzenat, Christian Meilicke, Heiner Stuckenschmidt, Pavel Shvaiko, and Cássia Trojahn. 2011. Ontology alignment evaluation initiative: Six years of experience. In *Journal on data semantics XV*, pages 158–192. Springer.

Daniel Faria, Catia Pesquita, Emanuel Santos, Matteo Palmonari, Isabel F Cruz, and Francisco M Couto. 2013. The AgreementMakerLight ontology matching system. In *On the Move to Meaningful Internet Systems (OTM)*, pages 527–541. Springer.

Guido Governatori, Antonino Rotolo, and Régis Riveret. 2018. A deontic argumentation framework based on deontic defeasible logic. In *Principles and Practice of Multi-Agent Systems (PRIMA)*, pages 484–492. Springer.

Guido Governatori and Meng Weng Wong. 2023. Defeasible semantics for L4. In *ProLaLa Workshop at POPL*.

David Graus. 2026. From legal text to executable decision models: Evaluating structured representations for legal decision model generation. In *Proceedings of the International Conference on Artificial Intelligence and Law (ICAIL)*. ACM.

Samyar Janatian, Hannes Westermann, Jinzhe Tan, Jaromir Savelka, and Karim Benyekhlef. 2023. From text to structure: Using large language models to support the development of legal expert systems. In *Legal Knowledge and Information Systems (JURIX 2023)*, pages 167–176.

Ernesto Jiménez-Ruiz and Bernardo Cuenca Grau. 2011. LogMap: Logic-based and scalable ontology matching. In *International Semantic Web Conference (ISWC)*, pages 273–288. Springer.

Arut Prakash Kaleeswaran, Arne Nordmann, Thomas Vogel, and Lars Grunske. 2022. A systematic literature review on counterexample explanation. *Information and Software Technology*, 145:106800.

Alan Khoja, Martin Kölbl, Stefan Leue, and Rüdiger Wilhelmi. 2025. Automated consistency analysis for legal contracts. *Artificial Intelligence and Law*.

A. Kuehlmann, V. Paruthi, F. Krohm, and M. K. Ganai. 2002. Robust Boolean reasoning for equivalence checking and functional property verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(12):1377–1394.

Zenan Li, Yifan Wu, Zhaoyu Li, Xinming Wei, Xian Zhang, Fan Yang, and Xiaoxing Ma. 2024. Autoformalize mathematical statements by symbolic equivalence and semantic consistency. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Megan Ma and Bryan Wilson. 2021. The legislative recipe: Syntax for machine-readable legislation. *Northwestern Journal of Technology and Intellectual Property*, 19(1):107.

Giulio Macilenti, Armando Stellato, and Manuel Fiorelli. 2024. Prompting is not all you need! Evaluating GPT-4's performance on a real-world ontology alignment use case. *Procedia Computer Science*, 246:1289–1298.

Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: a programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29.

Ezequiel José Veloso Ferreira Moreira and José Creissac Campos. 2025. On the role of generative AI in explaining model checking counterexamples. In *Engineering Interactive Computer Systems. EICS 2024 International Workshops*, volume 15518 of *Lecture Notes in Computer Science*, pages 138–158. Springer.

Ha Thanh Nguyen, Wachara Fungwacharakorn, Sabine Wehnert, May Myo Zin, Yuntao Kong, Jieying Xue, Michał Araszkiewicz, Randy Goebel, and Ken Satoh. 2026. GDPR auto-formalization with AI agents and human verification. In *Proceedings of the International Conference on Artificial Intelligence and Law (ICAIL)*. ACM.

Lorena Otero-Cerdeira, Francisco J. Rodríguez-Martínez, and Alma Gómez-Rodríguez. 2015. Ontology matching: A literature review. *Expert Systems with Applications*, 42(2):949–971.

Marcel Parciak, Brecht Vandevoort, Frank Neven, Liesbet M. Peeters, and Stijn Vansummeren. 2024. Schema matching with large language models: an experimental study. In *International Workshop on Tabular Data Analysis (TaDA)*.

W. V. Quine. 1952. The problem of simplifying truth functions. *The American Mathematical Monthly*, 59(8):521–531.

M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. 1986. The British Nationality Act as a logic program. *Communications of the ACM*, 29(5):370–386.

Shuncheng Tang, Zhenya Zhang, Jixiang Zhou, Lei Lei, Yuan Zhou, and Yinxing Xue. 2024. LeGEND: A top-down approach to scenario generation of autonomous driving systems assisted by large language models. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1497–1508.

A Models

Table 3 shows the models used for creating formalizations of the EU provisions.

Model	Provider
Claude Opus 4.6	Anthropic
GPT-5.4	OpenAI
Gemini 3.1 Pro	Google
DeepSeek-V3.2 (685B)	DeepSeek
Qwen 3.5 (397B)	Alibaba
Kimi K2.5 (1.1T)	Moonshot
Mistral Large 3 (675B)	Mistral
MiniMax M2.7 (229B)	MiniMax
GLM-5.1 (754B)	Zhipu AI

Table 3: LLMs used for formalization ($N = 9$).

B Worked Example

This example follows the steps of Section 4 for illustration. The provision and all data are constructed manually and are not drawn from the experimental data.

B.1 Provision

Figure 6 states the provision. The construction of paragraph 2 intentionally leaves open whether the two conditions for a deletion have to be fulfilled alternatively or cumulatively.

Data Retention

1. A controller may retain personal data where a legal basis for the processing exists and no ground for deletion applies.

2. The personal data has to be deleted if

(a) the retention period laid down in the controller's retention schedule has expired;

(b) the data subject has requested erasure of the data.

Figure 6: An intentionally ambiguous, fictitious legal provision.

B.2 Formalization

Figure 7 shows two formalizations of the provision, T_A and T_B. T_A interprets the norm such that either condition is sufficient to trigger the deletion, while T_B requires both conditions to be fulfilled. Additionally, T_B resolves the erasure request into the channels through which it can be made, where T_A leaves it atomic.

B.3 Matching

Matching (Section 4.2) groups the nodes that describe the same concept across the two formalizations; each such group is an *equivalence class* (EC). Here the five concepts the two formalizations share form one EC each, shown in Figure 8. The two channel nodes of T_B have no counterpart in T_A and therefore stay unmatched.

B.4 Interface

The deepest matched nodes are b, the legal basis, p, the elapsed period, and e, the erasure request. They form the interface $\mathcal{I}_{A,B} = \{b, p, e\}$ (Section 4.3.2). Note that e belongs to the interface although it is an internal node of T_B. Its two child nodes, describing the valid request channels, have no counterpart in T_A and therefore cannot be compared. They are ignored in the subsequent steps and e serves as an atomic variable.

All five nodes of T_A are on or above the interface, so $\mathrm{cov}(T_A, \mathcal{I}_{A,B}) = 5/5 = 1$. Of the seven nodes of T_B, five are on or above, so $\mathrm{cov}(T_B, \mathcal{I}_{A,B}) = 5/7 \approx 0.71$. The pairwise coverage is the geometric mean of the two, $\mathrm{cov}(T_A, T_B) = \sqrt{5/7} \approx 0.85$.

B.5 Edge Cases

Reading the matched nodes above the interface as operators over the interface variables (Sec-

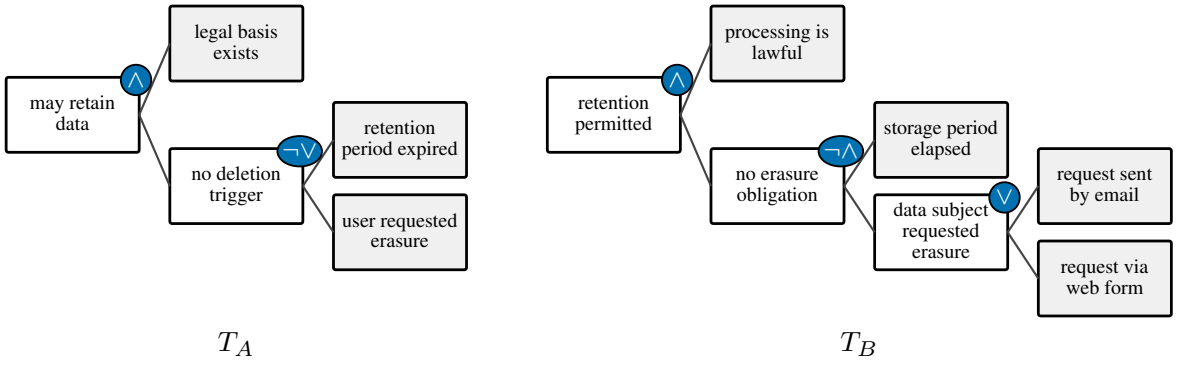


Figure 7: Two formalizations of the provision shown in Figure 6.

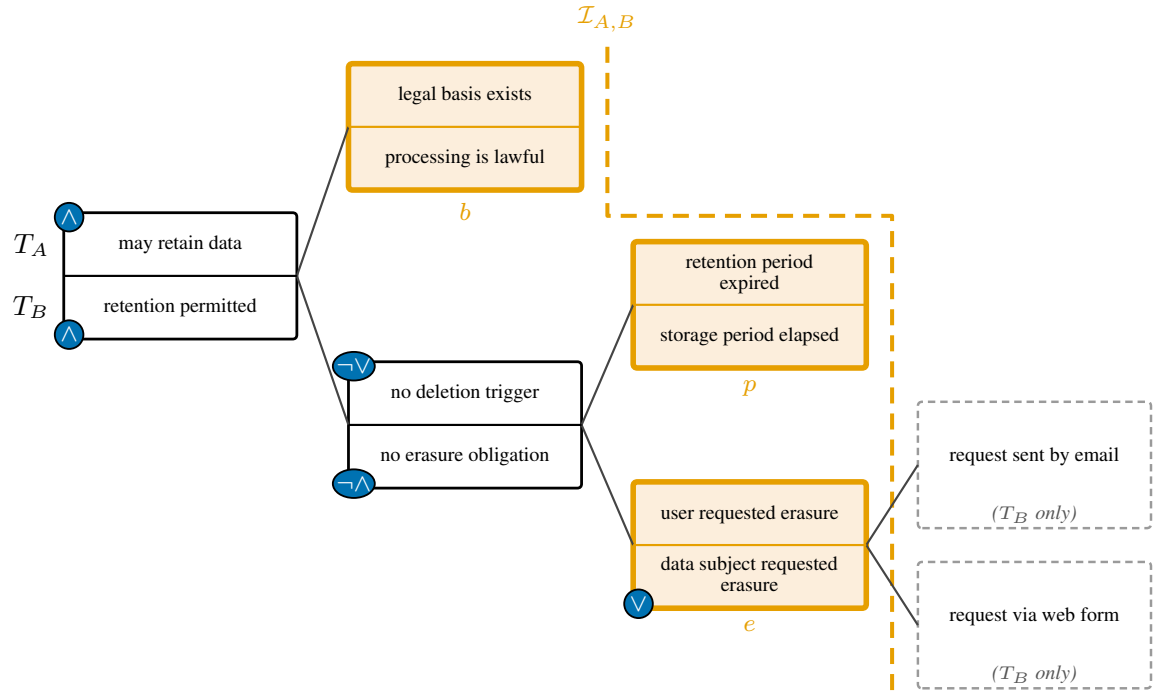


Figure 8: The two formalizations of Figure 7 after matching. The top half of each box is T_A, the bottom half T_B. The interface $\mathcal{I}_{A,B}$ (orange) holds the deepest nodes still matched to each other and supplies the Boolean variables b, p and e; it sits at different depths on different branches. Everything to the right of it belongs to T_B alone and is cut off.

tion 4.3.3) gives

$$f_A = b \wedge \neg(p \vee e),$$
$$f_B = b \wedge \neg(p \wedge e),$$

and hence $f_A \oplus f_B = b \wedge (p \oplus e)$. The disagreement function has exactly two prime implicants, $\{b, p, \neg e\}$ and $\{b, \neg p, e\}$, and the cover returned by the procedure of Appendix C contains both. The two formalizations reach opposite conclusions whenever a legal basis exists and exactly one of the two conditions of paragraph 2 is met.

Under either prime implicant, the deepest matched node whose two sub-formulas are forced to differ is the deletion node, which is therefore the root cause (Section 4.3.4).

B.6 Verbalization

Figure 9 shows an exemplary verbalization of one of the two prime implicants (Section 4.4). Applied to it, T_A concludes that the records may no longer be retained and T_B that they may.

Verbalization of $\{b, p, \neg e\}$

A hospital holds a patient's treatment records on a statutory legal basis. The retention period set out in its retention schedule expired last month. The patient has made no request for erasure.

May the hospital continue to retain the records?

T_A: no. T_B: yes.

Figure 9: A verbalization of one of the two edge cases.

C Edge Case Enumeration

We compute a non-redundant prime-implicant cover of the disagreement function $\varphi := f_A \oplus f_B$ by SAT-iterative search. The procedure maintains a growing set of *blocking clauses* β that excludes already-covered sub-cubes, and uses a second solver instance over $\neg\varphi$ to test the implicant property needed to prime a candidate cube. Figure 10 states the procedure; the steps below comment on each line. Literals are dropped in a fixed order (we sort the core by variable name), which makes the returned cover deterministic.

1. *Find an uncovered disagreement.* Check $\varphi \wedge \beta$, where β is the conjunction of the blocking clauses accumulated so far (initially empty, i.e. $\top$). If unsatisfiable, the cover is complete; return the accumulated cover. Otherwise let $\mathbf{m}$ be a returned satisfying assignment — i.e. a minterm of φ that no previously emitted prime implicant covers.

2. *Shrink to an implicant via unsat core.* Treating the literals of $\mathbf{m}$ as Z3 assumptions, check $\neg\varphi \wedge \mathbf{m}$. Since $\varphi(\mathbf{m}) = \texttt{true}$, this is unsatisfiable; the unsat core is a subset of the literals of $\mathbf{m}$ that already forces φ to hold under every completion of the remaining variables, hence an implicant of φ.

3. *Make the implicant prime by greedy literal drop.* Iterate over the literals of the core in a deterministic order; tentatively remove each one and verify (via a push/pop check on $\neg\varphi$ with the reduced cube as assumptions) that the remainder is still an implicant. Keep the literal otherwise. The result π is prime by construction.

4. *Block and iterate.* Add the clause $\neg\pi$ (i.e. the negation of the cube π) to β and return to step 1.

Input: $\varphi := f_A \oplus f_B$ over $\mathcal{I}_{A,B}$
Output: irredundant prime-implicant cover Π

$\Pi \leftarrow \emptyset$, $\beta \leftarrow \top$
while $\varphi \wedge \beta$ is SAT **do**
  $\mathbf{m} \leftarrow$ model of $\varphi \wedge \beta$
  $C \leftarrow \mathrm{unsatCore}(\neg\varphi, \mathbf{m})$
  $\pi \leftarrow \mathbf{m}|_C$
  for $\ell \in C$ in fixed order **do**
    if $\neg\varphi \wedge (\pi \setminus \ell)$ is UNSAT
      then $\pi \leftarrow \pi \setminus \ell$
  $\Pi \leftarrow \Pi \cup \{\pi\}$, $\beta \leftarrow \beta \wedge \neg\pi$
return Π

Figure 10: Edge-case enumeration. β accumulates one blocking clause $\neg\pi$ per emitted prime implicant, so each iteration searches only the part of φ no earlier implicant covers. $\mathbf{m}|_C$ is the minterm $\mathbf{m}$ restricted to the variables in the unsat core C.

Each iteration emits exactly one prime implicant and removes at least one previously uncovered minterm, so the loop terminates after at most $|\varphi^{-1}(\texttt{true})|$ steps. The resulting cover is irredundant by construction — every prime implicant eliminates a sub-cube that no prior one covered — but is not guaranteed to be of minimum cardinality, since the literal-drop order in step 3 is greedy rather than exhaustive. In practice we observe per-pair covers ranging from 1 to 58,283 prime implicants (Section 5.2).

D Case Study Provisions and Formalizations

For the two verbalized edge cases shown in Table 2, we reproduce the operative legal text and a side-by-side diagram of the two disagreeing formalizations. The diagrams use the same visual language as Figure 2: rounded boxes for ECs, operator badges for non-leaf nodes. The disagreement path is drawn left-to-right. Matched ECs are split by a horizontal divider into a top half (first formalization) and a bottom half (second), each carrying that formalization's verbatim label and its operator badge; unmatched nodes are single-sided and annotated below the label with the contributing formalization.

D.1 Art. 5 AI Act

Art. 5 AI Act — excerpt

1. The following AI practices shall be prohibited:
[. . .]
(g) the placing on the market, the putting into service for this specific purpose, or the use of biometric categorisation systems that categorise individually natural persons based on their biometric data to deduce or infer their race, political opinions, trade union membership, religious or philosophical beliefs, sex life or sexual orientation; this prohibition does not cover any labelling or filtering of lawfully acquired biometric datasets, such as images, based on biometric data or categorizing of biometric data in the area of law enforcement.
[. . .]

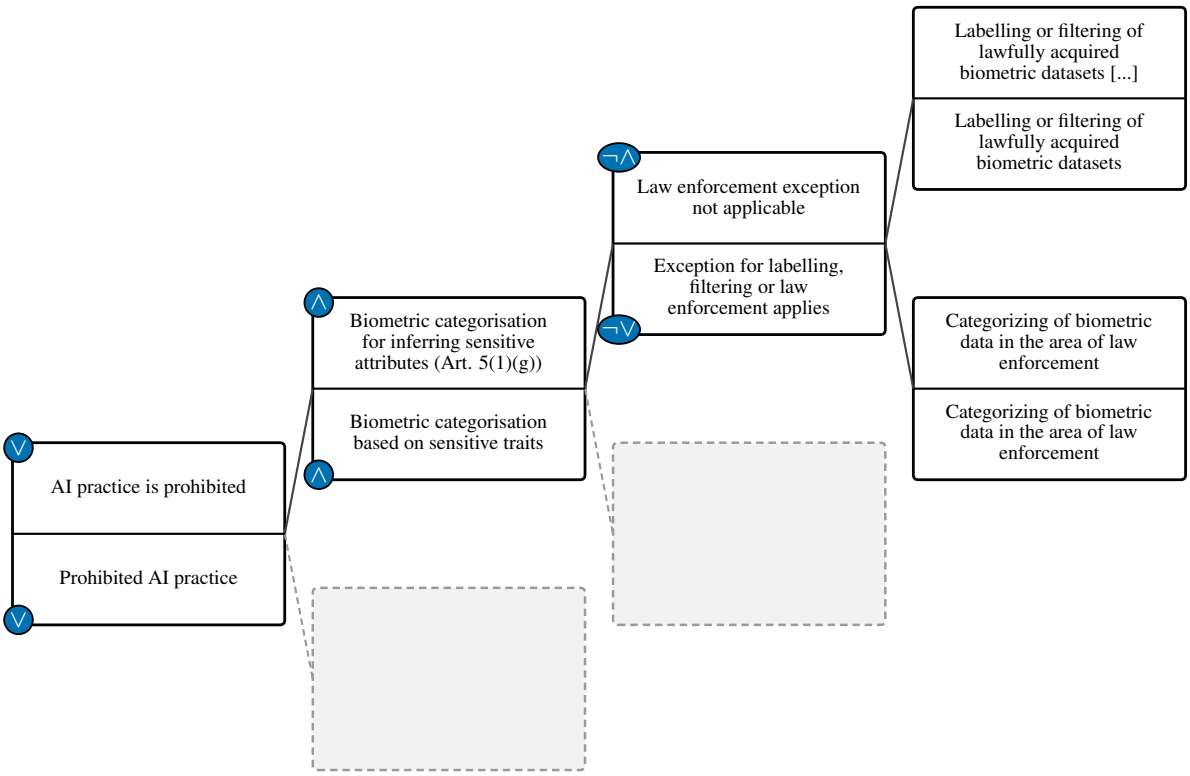


Figure 11: Matched formalizations of Art. 5(1)(g) AI Act by Claude Opus 4.6 (top half of each node) and Gemini 3.1 Pro (bottom half). The two formalizations agree on the high-level structure but diverge on the operator at "labelling/filtering exception not applicable": Claude treats the carve-out as NAND of conditions, Gemini as a NOR. Dashed grey nodes indicate unshown parts of the formalizations.

D.2 Art. 3 UCTD

Art. 3 UCTD (Directive 93/13/EEC)

1. A contractual term which has not been individually negotiated shall be regarded as unfair if, contrary to the requirement of good faith, it causes a significant imbalance in the parties' rights and obligations arising under the contract, to the detriment of the consumer.
2. A term shall always be regarded as not individually negotiated where it has been drafted in advance and the consumer has therefore not been able to influence the substance of the term, particularly in the context of a pre-formulated standard contract.
The fact that certain aspects of a term or one specific term have been individually negotiated shall not exclude the application of this Article to the rest of a contract if an overall assessment of the contract indicates that it is nevertheless a pre-formulated standard contract.
Where any seller or supplier claims that a standard term has been individually negotiated, the burden of proof in this respect shall be incumbent on him.
3. The Annex shall contain an indicative and non-exhaustive list of the terms which may be regarded as unfair.

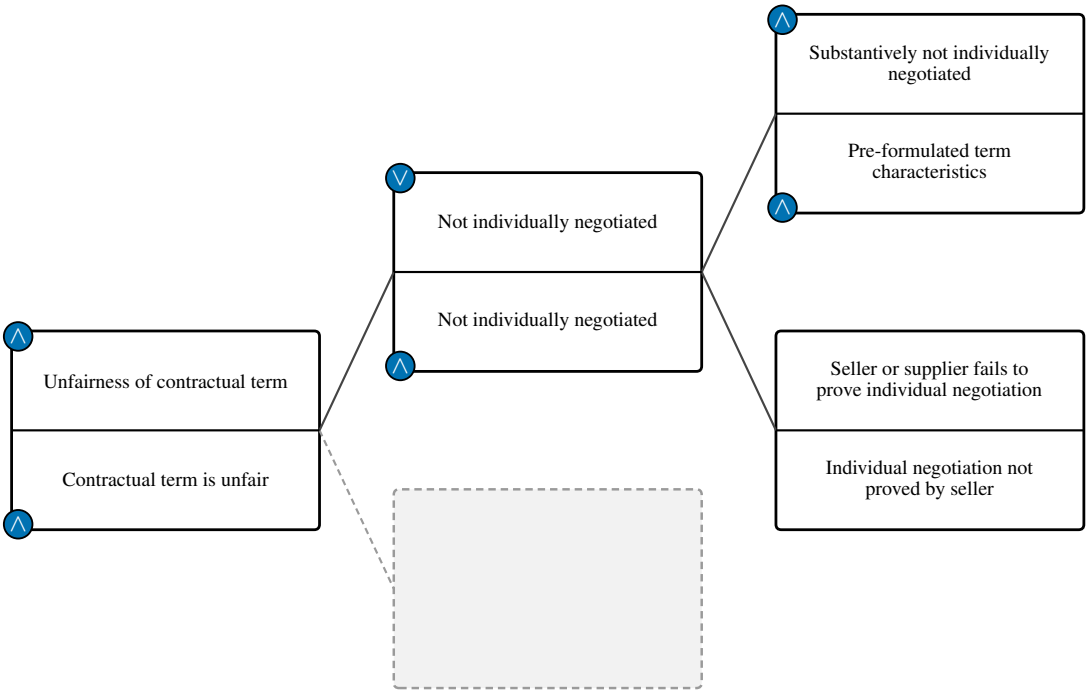


Figure 12: Matched formalizations of Art. 3 UCTD by GLM-5.1 (top half of each node) and Kimi-K2.5 (bottom half). The divergence sits at "not individually negotiated": GLM treats it as a disjunction (either pre-drafting or the burden-of-proof clause suffices), Kimi as a conjunction (both must hold). The dashed grey node indicates unshown parts of the formalizations. For clarity the visualization was slightly simplified.

E LLM Prompts

Figures 13–22 reproduce the prompt templates used in each stage of the pipeline, verbatim from the codebase. Placeholders enclosed in double curly braces (e.g. `{{SCHEMA}}`, `{{LAW_TEXT}}`) are substituted at runtime.

create_rulemap.md (part 1/3)

```
# Task

Create a Rulemap for the law text provided below.

Think of it as an assessment schema ("Prüfungsschema") a lawyer or judge would use:
1. You will be provided with the central legal consequence of the norm below
2. Build a decision tree a legal expert would use to determine whether that consequence applies
↪ in a specific case given the text of the law.

# Methodology

- **Do not** merely reframe the statutory wording. Use your knowledge of EU law and the
↪ relevant legal area to build the assessment schema as a domain expert would.
- If the law references **another law**, create a leaf node with the reference — do not resolve
↪ it.
- If the law references **another part of the same text** given to you, resolve that reference
↪ in the tree.
- The text of the law may contain parts that are not relevant for the central legal consequence
↪ provided to you. Ignore these parts of the law, but include *everything* that *is* relevant

# Structural Rules

1. **No single-child nodes.** Every non-leaf node must have at least two children.
2. **Granularity.**
  - Be as granular as necessary to assess individual cases, but not more.
  - Leaf nodes should be either
    - facts that are objectively assessable given a specific case or
    - common and general legal concepts that apply to that norm but are not specific to it
  - Focus on the matter specific to the norm to be formalized. If common and general legal
↪ concepts are to be assessed, do not resolve them in detail
3. **Labels.**
  - Use labels a legal domain expert would use as headings when assessing the law
  - Do not cite authors in the labels
  - Do not make comments in the label about reasons for your choice or any other meta-comments
↪ on the label
  - Do not put long enumerations in the label, but use a short wording that summarizes the set
↪ of cases precisely.
4. **IDs.** Assign sequential integer IDs starting at 1 (root = 1).
5. **Name.** The top-level `name` field should be a short identifier for the law (e.g. `"Art.
↪ 101 TFEU"` or `"GDPR Art. 6"`).
```

Figure 13: Formalization stage (part 1/3): task description, methodology, and structural rules.

create_rulemap.md (part 2/3)

```
## Commented examples
  1. Theft
    - Bad:
      - Suspect took away movable object belonging to another person [COMMENT: these labels are
↪ mere rewordings of the law text and don't make the established legal concepts explicit.]
        - object was movable
        - object belongs to another person
        - Suspect took away the object (as defined by Schneider) [COMMENT: Do not make any
↪ comments on your label choices.]
    - Good:
      - Movable object belonging to another
        - Movable object
        - Belonging to another
      - Taking  [COMMENT: "Taking" is the established legal category to be checked when
↪ assessing Theft, thus it has to be part of the structure]
        - Custody of another person [COMMENT: "Custody" is another concept in the context of
↪ Theft, thus it must appear here although it is not named explicitely in the text of the
↪ law.]
        - Breaking of custody
          - Ending of original custody
          - against the will
        - Establishment of new custody
  2. Sales contract
    - Bad:
      - Parties closed a sales contract
        - Party 1 made an offer
        - Party 2 accepted the offer
        - Offer and acceptance are about the sale of an object
        - Offer and acceptance match each other (ambigious, but mostly accepted) [COMMENT: Do
↪ not comment on characteristics of label or why you decided to choose it.]
        - No invalidity due to lack of legal capacity, avoidance, immorality, ursury or
↪ violation of statutory prohibitions [COMMENT: Do not put long enumerations in labels.]
          - lack of legal capacity [COMMENT: Reasons for invalidity are common to most
↪ contracts and not specifically related to sales contracts. Resolving them here is beyond
↪ the scope of the formalization.]
            - age < 7 years
            - ...
          - usury
            - ...
          - ...
    - Good:
      - Sales contract
        - offer
        - acceptance
        - Contract main content: Sale
        - No invalidity
```

Figure 14: Formalization stage (part 2/3): commented examples illustrating good and bad node labels and granularity.

create_rulemap.md (part 3/3)

```
# Node Properties

- **`operator`** (required for every node):
  - `"and"` — all children must be satisfied.
  - `"or"` — at least one child must be satisfied.
  - `"virtual"` — **leaf nodes only**; indicates no logical operator (terminal condition).
- **`negation`** (`boolean`, default `false`):
  - An operator can be logically negated by setting negation to 'true'.
  - That is, nodes with `operator` = `and`/`or` and `negation` = `true` are interpreted as
↪ `nand` and `nor`, respectively.
  - If you decide to set `negation` = `false`, you may also omit this field.

**`operator`** and **`negation`** define the logical operator each non-leaf nodes represents.
↪ Remember that the final rule map will be used to assess actual legal cases. Therefore, make
↪ sure that the values you set for **`operator`** and **`negation`**
align with your legal interpretation of the norm and the operators of the other nodes.

**`children`** is a required field. If a node is a leaf node, still add the field with an empty
↪ list \[\] as value.


# Output Format

- Return **ONLY valid JSON**. No markdown formatting, no preamble, no explanations.
- The JSON must comply with the following schema (do not include the schema meta-properties in
↪ your output):

{{SCHEMA}}

# Law Text

{{LAW_TEXT}}

# Central legal consequence

{{LEGAL_CONSEQUENCE}}
```

Figure 15: Formalization stage (part 3/3): node properties, output format, and runtime placeholders.

create_rulemap_correction.md

```
Your previous output failed validation. The error report is below.

## How to read the error report

The validator performs two checks:

1. **JSON Schema validation** — Your output must conform to the rulemap schema. Common schema
↪ errors include:
   - Missing required fields (`name`, `root`, `id`, `label`, `children`)
   - Wrong types (e.g., `id` must be an integer, `operator` must be one of `"and"`, `"or"`,
↪ `"virtual"`)
   - Invalid `operator` values or extra properties not allowed by the schema
   - Leaf nodes must have `children: []` (an empty array, not omitted)

2. **Unique ID check** — Every node `id` in the tree must be unique. If the same integer ID
↪ appears on multiple nodes, you will see: `Duplicate id detected in rulemap: <id>`.

If the error says "Expected a JSON object", your response was not valid JSON or was wrapped in
↪ markdown formatting. Return **only** the raw JSON object.

## Error report

{{ERROR}}

## Instructions

Fix the issues described above and return only the corrected, valid JSON. Do not include
↪ markdown code fences, explanations, or any text outside the JSON object.
```

Figure 16: Formalization stage: follow-up prompt sent when the LLM's output fails JSON schema validation.

matching.md

```
__Input:__

1. __N Rule Maps (JSON):__ Each is a tree-structured formalization of the same legal norm,
↪ created independently. Nodes represent legal conditions connected by logical operators
↪ (`and`, `or`, `virtual`).
2. __Norm Text:__ The actual legal text to resolve ambiguities.

__Task:__ Group the nodes from all N rule maps into __equivalence classes__ (ECs). Each EC
↪ groups nodes across rule maps that represent the same legal concept. Nodes without
↪ counterparts in other rule maps form singleton ECs.

__Matching Strategy:__

Match nodes based on their conceptual legal role in the respective trees. Refer to the text of
↪ the law to identify the legal meaning of the nodes in the context of the whole structure of
↪ each tree.
If a node has no counterpart in any other tree (e.g., because of differing granularity or
↪ entirely different structuring), place it in a __singleton EC__ (an EC with only one
↪ member).

__Mandatory Rules:__

1. __At most one node per rule map per EC:__ Each EC may contain at most one node from each
↪ rule map. This means no two nodes from the same rule map can be in the same EC.

2. __Exhaustiveness:__ Every node from every rule map must appear in exactly one EC. No node
↪ may be omitted or appear in multiple ECs.

3. __Roots matched:__ The root nodes of all rule maps must be in the same EC.

4. __EC Labels:__ Each EC must have a descriptive `label` field that summarizes the legal
↪ concept it represents (e.g., "Applicability of Article 101 TFEU", "undertaking or
↪ association of undertakings", "agreement between undertakings"). This label should reflect
↪ the common legal meaning, not just copy a single node's label.

5. __No cycles:__ The parent-child relationships derived from the tree structures must form a
↪ directed acyclic graph (DAG) when projected onto the ECs. Concretely: if node A is an
↪ ancestor of node B in one tree, then the EC containing A must not be a descendant of the EC
↪ containing B in any other tree's structure.

6. __Schema Compliance:__

   - Your result __must__ conform to the following JSON schema:
   `{{SCHEMA}}`

7. __No Hallucinations:__ You must use the exact node objects (ID, Label, Operator) provided in
↪ the input. Do not invent new IDs or labels. Copy label and operator strings exactly as
↪ given. For `rulemap_id`: each rule map is presented with a header `=== File N:
↪ filename.json ===`. The `rulemap_id` is the **last underscore-separated segment** of the
↪ filename (without `.json`). Examples: `9_gdpr_anthropic_claude-opus-4-6.json` →
↪ `claude-opus-4-6`; `9_gdpr_ollama_minimax-m2.7.json` → `minimax-m2.7`.

8. __Output Format:__ Return __ONLY valid JSON__. No markdown formatting (like ```json), no
↪ preamble, no explanations.

# Law Text

{{LAW_TEXT}}

## Rule Maps

{{RULEMAPS}}
```

Figure 17: Matching stage: prompt for N-way matching of nodes across rule maps.

matching_correction.md

```
Your previous output failed validation. The error report is below.

## How to read the error report

The validator performs a JSON Schema check followed by 7 integrity checks:

1. **JSON Schema validation** — The output must match the N-way matching schema. Common errors:
↪ missing `label` or `members` in an equivalence class, empty `members` array, wrong field
↪ types.

2. **Rulemap ID uniqueness** — Each `rulemap_id` referenced in members must match one of the
↪ input rule map filenames. If you see "invalid rulemap ref", you used a filename that
↪ doesn't exist in the input.

3. **One node per rulemap per EC** — Each rule map may contribute at most one node to each
↪ equivalence class. If you see a "collision" error, the same `rulemap_id` appears twice in
↪ one EC's members.

4. **Node coverage** — Every node from every input rule map must appear in exactly one
↪ equivalence class.
   - "missing nodes" = nodes from the input that you didn't assign to any EC.
   - "hallucinated nodes" = node IDs you referenced that don't exist in the input rule maps.
   - "duplicate nodes" = the same node appears in multiple ECs.

5. **Label consistency** — The `label` and `operator` you report for each member must match
↪ the actual values in the source rule map. Mismatches indicate you copied labels incorrectly.

6. **Roots in same EC** — All root nodes (the top-level node of each rule map) must be grouped
↪ into the **same** equivalence class.

7. **DAG check** — The parent-child relationships implied by the ECs must form a directed
↪ acyclic graph. If you see a "cycle" error, the grouping creates circular dependencies.

## Error report

{{ERROR}}

## Instructions

Fix the issues described above and return only the corrected, valid JSON. Do not include
↪ markdown code fences, explanations, or any text outside the JSON object.
```

Figure 18: Matching stage: follow-up prompt sent when the matching output fails validation.

verbalize_edge_case.md (part 1/3)

```
# Task: Verbalize a Formal Edge Case into a Concrete Legal Scenario

Multiple LLM-generated rule maps for the same EU legal provision disagree on
whether the provision **applies** under certain input conditions. SAT analysis
has identified a *minimal* fact pattern (the **prime implicant**) that forces
the disagreement: any scenario matching these facts produces the disagreement,
regardless of what other facts are true.

Your job is to write a short, realistic scenario illustrating exactly this fact
pattern, pose the applicability question it provokes, and explain how the two
rule maps disagree as a question of *legal interpretation* — so a human reader
can judge which interpretation is more faithful to the provision.

## Inputs

You are given the raw formal data below. No prose rendering has been applied;
you must read the JSON objects and do all narrative work yourself.

### 1. Legal provision

{{PROVISION_TEXT}}

### 2. Projected EC graph (two rule maps)

A directed graph of *equivalence classes* (ECs) restricted to the two rule
maps in this disagreement. Every EC has an `id`, a `canonical_label`, and a
`members` list — one entry per rule map that contains this EC, recording the
logical `operator` (`and` / `or` / `virtual`) and `negation` at that node.
Edges carry `supporting_rulemaps`: which of the two rule maps uses that
parent → child link.

A rule map's tree for this provision is reconstructed by walking the edges
filtered on its `rulemap_id` and reading the operator off each EC's
corresponding member entry. A `virtual` operator marks a leaf (atomic
condition).

```json
{{PROJECTED_EC_GRAPH}}
```
```

Figure 19: Verbalization stage (part 1/3): task description and inputs (legal provision, projected EC graph).

verbalize_edge_case.md (part 2/3)

```
### 3. Stipulated facts (prime implicant)

Each entry is an EC-graph node augmented with `"value": true|false`. These
assignments **must hold** in the scenario you construct. Together they force
the two rule maps to disagree; no other facts are needed.

```json
{{STIPULATED_PI}}
```

4. Sides of the disagreement

- Rule map concluding the provision **applies**: `{{RM_TRUE}}`
- Rule map concluding the provision **does not apply**: `{{RM_FALSE}}`

5. Deepest forced-divergence points (root causes)

Raw symbolic records for the ECs where the structural divergence is deepest
(one entry per root cause). Each dict carries `ec_id`, `ec_label`, and
per-side operator / negation / children information. Use these to locate
the disagreement inside the projected graph.

```json
{{ROOT_CAUSES}}
```

Instructions

Produce a single JSON object with the following fields.

`title`

A short (<= 10 words) descriptive title for the scenario.
```

Figure 20: Verbalization stage (part 2/3): inputs continued (stipulated facts, sides of the disagreement, deepest forced-divergence points) and instructions for the title field.

verbalize_edge_case.md (part 3/3)

```
### `scenario`

A factual narrative (2-6 sentences) describing a plausible real-world
situation in which every stipulated fact holds with the indicated
`true` / `false` value. Hard rules:

- **Facts only.** State what the actors did and what the factual
  circumstances are. Do *not* state, imply, or evaluate legal conclusions
  (e.g. do not write "the clause is unfair", "the processing is unlawful",
  "the exception applies"). Whether the provision applies is exactly the
  contested question, it must not be pre-judged by the narrative.
- **No parentheticals.** Do not insert inline citations or clarifications in
  parentheses or brackets such as "(Art. 6(1)(f) GDPR)", "(legitimate
  interest)", "[i.e. ...]". Legal citations belong in the legal provision
  text, not in the scenario. If a factual label is needed, integrate it as
  plain prose.
- **Concrete.** Use realistic parties, data, and circumstances. Avoid
  abstract placeholders.
- **Minimal.** Omit narrative detail that is not required to establish a
  stipulated fact.
- **Rely on plausibility for the default case.** Do not state a stipulated
  fact explicitly when its value is what any reasonable reader would already
  assume from the other facts in the scenario.  Example: if the scenario
  describes an eight-year-old child, you do *not* need to write
  "the child is unmarried" to establish a stipulated ``married = false``.
  Unmarried is the default the reader will infer, and stating it is awkward.
  State the fact explicitly *only* when its stipulated value contradicts
  the default expectation.

### `question`

One short neutral question (<= 25 words) asking whether the provision applies
to this scenario. Do not hint at an answer.

### `divergence_analysis`

A concise legal-interpretation contrast between the two sides. Explain, in
legal terms, *what interpretive question* divides the two formalizations on
this fact pattern and *why each reading is defensible* given the provision
text. Focus on the substantive legal disagreement — the contested scope,
condition, or exception — not the formal structure.

Do not describe the rule-map formalism, operators, or EC-graph structure.
Do not use "AND", "OR", "NOT-OR", "negation", "EC IDs", "virtual nodes",
"prime implicants", "equivalence classes", or similar technical vocabulary.
A reader who has never seen a rule map must be able to follow it.

### `interpretation_type`

Classify the disagreement as one of:
- `"legal_ambiguity"` — the provision text is genuinely open to both
  readings; a reasonable legal expert could defend either.
- `"formalization_error"` — one reading appears to misparse the
  provision (e.g. treats disjunctive conditions as conjunctive, misreads
  scope of an exception, omits a required element).

## Output format

Return **only** the JSON object, with no markdown fences, no preamble, and no
trailing commentary.
```

Figure 21: Verbalization stage (part 3/3): instructions for the remaining output fields, and output format.

verbalize_correction.md

```
Your previous response failed validation.  The validator reports:

{{ERROR}}

## Instructions

Fix the issues above and return a corrected JSON object that conforms to
the schema.  Do not include markdown code fences, explanations, or any
text outside the JSON object itself.
```

Figure 22: Verbalization stage: follow-up prompt sent when the verbalization output fails validation.